Introduction

Structural design patterns are concerned with the **composition of classes and objects**. They focus on how to assemble classes and objects into larger structures while keeping these structures flexible and efficient. Decorator Pattern is one of the most important structural design patterns. Let's understand in depth.

Decorator Pattern

The **Decorator Pattern** is a **structural design pattern** that allows behavior to be added to individual objects, dynamically at runtime, without affecting the behavior of other objects from the same class.

It wraps an object inside another object that adds new behaviors or responsibilities at runtime, keeping the original object's interface intact.

Real-Life Analogy

Think of a coffee shop:

- You order a simple coffee.
- Then, you can add milk, add sugar, add whipped cream, etc.
- You don't need a whole new drink class for every combination.

Each addition wraps the original and adds something more.

Problem It Solves

It solves the problem of **class explosion** that occurs when you try to use inheritance to add combinations of behavior. **For Example, imagine you have:**

- A Pizza

- A CheesePizza
- A CheeseAndOlivePizza
- A CheeseAndOliveStuffedPizza

Every combination would need a new subclass as shown in the code below.

```
#include <bits/stdc++.h>

using namespace std;


// Each combination of pizza requires a new class

class PlainPizza {};

class CheesePizza : public PlainPizza {};

class OlivePizza : public PlainPizza {};

class StuffedPizza : public PlainPizza {};

class CheeseStuffedPizza : public CheesePizza {};

class CheeseOlivePizza : public CheesePizza {};

class CheeseOliveStuffedPizza : public CheeseOlivePizza {};


int main() {

   // Base pizza

   PlainPizza plainPizza;
```

```
    // Pizzas with individual toppings
    CheesePizza cheesePizza;
    OlivePizza olivePizza;
    StuffedPizza stuffedPizza;

    // Combinations of toppings require separate classes
    CheeseStuffedPizza cheeseStuffedPizza;
    CheeseOlivePizza cheeseOlivePizza;

    // Further combinations increase complexity exponentially
    CheeseOliveStuffedPizza cheeseOliveStuffedPizza;

    return 0;
}

import java.util.*;

// Each combination of pizza requires a new class
class PlainPizza {}
class CheesePizza extends PlainPizza {}
class OlivePizza extends PlainPizza {}
```

```
class StuffedPizza extends PlainPizza {}

class CheeseStuffedPizza extends CheesePizza {}

class CheeseOlivePizza extends CheesePizza {}

class CheeseOliveStuffedPizza extends CheeseOlivePizza {}


public class Main {
   public static void main(String[] args) {
      // Base pizza
      PlainPizza plainPizza = new PlainPizza();

      // Pizzas with individual toppings
      CheesePizza cheesePizza = new CheesePizza();
      OlivePizza olivePizza = new OlivePizza();
      StuffedPizza stuffedPizza = new StuffedPizza();

      // Combinations of toppings require separate classes
      CheeseStuffedPizza cheeseStuffedPizza = new CheeseStuffedPizza();
      CheeseOlivePizza cheeseOlivePizza = new CheeseOlivePizza();

      // Further combinations increase complexity exponentially
      CheeseOliveStuffedPizza cheeseOliveStuffedPizza = new
CheeseOliveStuffedPizza();
```

```
    }
}
# Each combination of pizza requires a new class
class PlainPizza:
    pass

class CheesePizza(PlainPizza):
    pass

class OlivePizza(PlainPizza):
    pass

class StuffedPizza(PlainPizza):
    pass

class CheeseStuffedPizza(CheesePizza):
    pass

class CheeseOlivePizza(CheesePizza):
    pass
```

```
class CheeseOliveStuffedPizza(CheeseOlivePizza):
    pass



if __name__ == "__main__":
    # Base pizza
    plainPizza = PlainPizza()

    # Pizzas with individual toppings
    cheesePizza = CheesePizza()
    olivePizza = OlivePizza()
    stuffedPizza = StuffedPizza()

    # Combinations of toppings require separate classes
    cheeseStuffedPizza = CheeseStuffedPizza()
    cheeseOlivePizza = CheeseOlivePizza()

    # Further combinations increase complexity exponentially
    cheeseOliveStuffedPizza = CheeseOliveStuffedPizza()
```

This quickly becomes unmanageable. Here, the Decorator Pattern comes into play. It lets you **compose behaviors** using wrappers instead of subclassing.

Solution to Pizza Problem

The Decorator Pattern solves the above discussed **Pizza problem**. It allows us to add responsibilities (like toppings) to objects dynamically without modifying their structure.

Instead of relying on a rigid class hierarchy, we compose objects using wrappers. This promotes flexibility, scalability, and cleaner code architecture.

Using Decorator Pattern

```
#include <bits/stdc++.h>

using namespace std;


// ========== Component Interface ==========

class Pizza {

public:

   virtual string getDescription() = 0;

   virtual double getCost() = 0;

   virtual ~Pizza() = default;

};
```

```
// ============= Concrete Components: Base pizza ==============
class PlainPizza : public Pizza {
public:
    string getDescription() override {
        return "Plain Pizza";
    }

    double getCost() override {
        return 150.00;
    }
};

class MargheritaPizza : public Pizza {
public:
    string getDescription() override {
        return "Margherita Pizza";
    }

    double getCost() override {
        return 200.00;
```

```
    }
};

// ========================= Abstract Decorator ==========================

// ====== Implements Pizza and holds a reference to a Pizza object =======

class PizzaDecorator : public Pizza {
protected:
    Pizza* pizza;
public:
    PizzaDecorator(Pizza* pizza) : pizza(pizza) {}
    virtual ~PizzaDecorator() { delete pizza; }
};

// ============ Concrete Decorator: Adds Extra Cheese ================

class ExtraCheese : public PizzaDecorator {
public:
    ExtraCheese(Pizza* pizza) : PizzaDecorator(pizza) {}

    string getDescription() override {
```

```
        return pizza->getDescription() + ", Extra Cheese";
    }

    double getCost() override {
        return pizza->getCost() + 40.0;
    }
};

// ============ Concrete Decorator: Adds Olives ================
class Olives : public PizzaDecorator {
public:
    Olives(Pizza* pizza) : PizzaDecorator(pizza) {}

    string getDescription() override {
        return pizza->getDescription() + ", Olives";
    }

    double getCost() override {
        return pizza->getCost() + 30.0;
    }
};
```

```
// ========== Concrete Decorator: Adds Stuffed Crust Cheese ==========

class StuffedCrust : public PizzaDecorator {

public:

  StuffedCrust(Pizza* pizza) : PizzaDecorator(pizza) {}


  string getDescription() override {

    return pizza->getDescription() + ", Stuffed Crust";

  }


  double getCost() override {

    return pizza->getCost() + 50.0;

  }

};




// Driver code

int main() {

  // Start with a basic Margherita Pizza

  Pizza* myPizza = new MargheritaPizza();
```

```
    // Add Extra Cheese
    myPizza = new ExtraCheese(myPizza);

    // Add Olives
    myPizza = new Olives(myPizza);

    // Add Stuffed Crust
    myPizza = new StuffedCrust(myPizza);

    // Final Description and Cost
    cout << "Pizza Description: " << myPizza->getDescription() << endl;
    cout << "Total Cost: ₹" << myPizza->getCost() << endl;

    delete myPizza;
    return 0;
}

import java.util.*;

// ========== Component Interface ==========
interface Pizza {
```

```
    String getDescription();
    double getCost();
}

// ============= Concrete Components: Base pizza ==============
class PlainPizza implements Pizza {
    @Override
    public String getDescription() {
        return "Plain Pizza";
    }

    @Override
    public double getCost() {
        return 150.00;
    }
}

class MargheritaPizza implements Pizza {
    @Override
    public String getDescription() {
        return "Margherita Pizza";
```

```
    }

    @Override
    public double getCost() {
        return 200.00;
    }
}

// ========================== Abstract Decorator ============================

// ====== Implements Pizza and holds a reference to a Pizza object =======
abstract class PizzaDecorator implements Pizza {
    protected Pizza pizza;

    public PizzaDecorator(Pizza pizza) {
        this.pizza = pizza;
    }
}
```

```
// ============ Concrete Decorator: Adds Extra Cheese ================

class ExtraCheese extends PizzaDecorator {

    public ExtraCheese(Pizza pizza) {

        super(pizza);

    }


    @Override

    public String getDescription() {

        return pizza.getDescription() + ", Extra Cheese";

    }


    @Override

    public double getCost() {

        return pizza.getCost() + 40.0;

    }

}


// ============ Concrete Decorator: Adds Olives ================

class Olives extends PizzaDecorator {

    public Olives(Pizza pizza) {
```

```
        super(pizza);
    }

    @Override
    public String getDescription() {
        return pizza.getDescription() + ", Olives";
    }

    @Override
    public double getCost() {
        return pizza.getCost() + 30.0;
    }
}

// ========== Concrete Decorator: Adds Stuffed Crust Cheese ==========
class StuffedCrust extends PizzaDecorator {
    public StuffedCrust(Pizza pizza) {
        super(pizza);
    }

    @Override
```

```
    public String getDescription() {
        return pizza.getDescription() + ", Stuffed Crust";
    }

    @Override
    public double getCost() {
        return pizza.getCost() + 50.0;
    }
}


// Driver code
public class Main {
    public static void main(String[] args) {
        // Start with a basic Margherita Pizza
        Pizza myPizza = new MargheritaPizza();

        // Add Extra Cheese
        myPizza = new ExtraCheese(myPizza);

        // Add Olives
```

```
        myPizza = new Olives(myPizza);

        // Add Stuffed Crust
        myPizza = new StuffedCrust(myPizza);

        // Final Description and Cost
        System.out.println("Pizza Description: " + myPizza.getDescription());
        System.out.println("Total Cost: ₹" + myPizza.getCost());
    }
}
```

```
from abc import ABC, abstractmethod

# ========== Component Interface ==========
class Pizza(ABC):
    @abstractmethod
    def getDescription(self):
        pass

    @abstractmethod
    def getCost(self):
```

```
        pass



# ============= Concrete Components: Base pizza ==============

class PlainPizza(Pizza):
    def getDescription(self):
        return "Plain Pizza"

    def getCost(self):
        return 150.00



class MargheritaPizza(Pizza):
    def getDescription(self):
        return "Margherita Pizza"

    def getCost(self):
        return 200.00
```

```
# ========================= Abstract Decorator ==========================

# ====== Implements Pizza and holds a reference to a Pizza object =======

class PizzaDecorator(Pizza):

    def __init__(self, pizza):

        self.pizza = pizza



# ============ Concrete Decorator: Adds Extra Cheese ================

class ExtraCheese(PizzaDecorator):

    def getDescription(self):

        return self.pizza.getDescription() + ", Extra Cheese"


    def getCost(self):

        return self.pizza.getCost() + 40.0



# ============ Concrete Decorator: Adds Olives ================

class Olives(PizzaDecorator):

    def getDescription(self):
```

```
        return self.pizza.getDescription() + ", Olives"

    def getCost(self):
        return self.pizza.getCost() + 30.0


# ============ Concrete Decorator: Adds Stuffed Crust Cheese ================
class StuffedCrust(PizzaDecorator):
    def getDescription(self):
        return self.pizza.getDescription() + ", Stuffed Crust"

    def getCost(self):
        return self.pizza.getCost() + 50.0


# Driver code
if __name__ == "__main__":
    # Start with a basic Margherita Pizza
    myPizza = MargheritaPizza()

    # Add Extra Cheese
```

```
myPizza = ExtraCheese(myPizza)


# Add Olives

myPizza = Olives(myPizza)


# Add Stuffed Crust

myPizza = StuffedCrust(myPizza)


# Final Description and Cost

print("Pizza Description:", myPizza.getDescription())

print("Total Cost: ₹" + str(myPizza.getCost()))
```

Understanding the Code

The above code:

- Defines a `Pizza` interface that all pizzas (base and decorated) must implement.
- Implements two concrete `PlainPizza` and `MargheritaPizza` as the base pizzas.
- Defines an abstract `PizzaDecorator` which wraps a `Pizza` object and forwards method calls to it.
- Implements concrete decorators like `ExtraCheese`, `Olives`, and `StuffedCrust` which extend the functionality of the pizza object.
- In the `main` method:
    - A plain Margherita pizza is created.

    - It is then wrapped successively with different decorators: `ExtraCheese`, `Olives`, and `StuffedCrust`.
    - Each decorator adds to the pizza's description and cost.
    - Finally, the composed pizza's description and total cost are printed.

How Decorator Pattern Solves the Issue

- **Avoids Class Explosion:** You no longer need a separate class for each combination of toppings. Just create new decorators as needed.
- **Flexible & Scalable:** Toppings can be added, removed, or reordered at runtime, offering high customization.
- **Follows Open/Closed Principle:** The base `Pizza` classes are open for extension (via decorators) but closed for modification.
- **Cleaner Code Architecture:** Composition is used instead of inheritance, resulting in loosely coupled components.
- **Promotes Reusability:** Each topping is a self-contained decorator and can be reused across different pizza compositions.

Key Takeaways

- **Abstract Classes and Constructors:**
  Abstract classes can have constructors, and these constructors are executed when a subclass is instantiated. This is important for initializing common properties or behavior shared across all subclasses.
- **Decorator as Layers:**
  Each decorator acts like a layer, similar to wrapping a gift box. Every decorator adds behavior on top of the previous one, allowing for flexible and dynamic composition of functionality.
- **Call Stack Analogy:**
  The Decorator Pattern functions like a call stack, where behavior is

accumulated step by step as each decorator wraps the component. This stacked behavior is then unwrapped during method calls, preserving the order and layering.

- **Loose Coupling Between Classes:** The use of interfaces and composition in the Decorator Pattern ensures loose coupling between components. This makes the system more flexible, testable, and easier to extend without affecting existing code.

When Should You Use the Decorator Pattern?

The **Decorator Pattern** is particularly useful in scenarios where flexibility, modularity, and extensibility are key. Consider using it when:

- **You need to add responsibilities to objects dynamically:**
  Instead of hardcoding behaviors into a class, decorators allow you to attach additional functionality at runtime, offering great flexibility.
- **You want to avoid an explosion of subclasses:**
  For every possible combination of features, creating separate subclasses leads to unmanageable and bloated class hierarchies. Decorators eliminate this by composing behaviors.
- **You want to follow the Open/Closed Principle (OCP):**
  The pattern supports the OCP by allowing classes to be open for extension but closed for modification. You enhance behavior without altering existing code.
- **You want reusable and composable behaviors:**
  Decorators can be reused across different components and can be composed in various combinations to achieve desired functionality.
- **You need layered, step-by-step enhancements:**
  Decorators can be applied one after another, layering features gradually in a controlled and traceable way—much like wrapping layers around an object.

Advantages

A few advantages of using the Decorator Pattern are:

- **Adheres to the Open/Closed Principle (OCP):** Enhancements can be made without modifying existing code, supporting scalability and maintainability.
- **Runtime Flexibility to Compose Features:** Behaviors can be added or removed dynamically, allowing for highly customizable solutions.
- **Avoids Subclass Explosion:** Instead of creating multiple subclasses for every feature combination, decorators provide a cleaner, more modular approach.
- **Promotes Single Responsibility for Each Add-on:** Each decorator focuses on a specific functionality, leading to better code organization and readability.

Disadvantages

A few trade-offs while using the Decorator Pattern are:

- **Can Result in Many Small Classes:** Each feature typically requires its own decorator class, which can clutter the codebase.
- **Stack Trace Debugging is Difficult:** Debugging layered decorators can be challenging, as stack traces may become complex and harder to trace.
- **Overhead of Multiple Wrapping Classes:** Composing many decorators can introduce runtime overhead and make the class structure harder to follow.
- **Developers Must Understand Decorator Flow:** Proper implementation requires developers to grasp the decorator chaining logic, which may introduce a learning curve.

Real-World Use Cases

The **Decorator Pattern** is widely used in real-life software products to enable dynamic behavior composition without bloating the class hierarchy. Below are practical examples where it plays a critical role:

1. Food Delivery Applications (e.g., Swiggy, Zomato)

Context: Customers can customize food items with add-ons like extra cheese, sauces, toppings, or side dishes.

Role of Decorator Pattern:

- Each add-on modifies the base food item's description and price dynamically.
- Instead of creating subclasses for every combination (e.g., `PizzaWithCheeseAndOlives`), decorators like `CheeseDecorator`, `OliveDecorator`, etc., can be stacked over a base `Pizza`.
- This allows the system to stay open for extension (new add-ons) but closed for modification.

2. Google Docs or Word Processors

Context: Users can apply text formatting like bold, italic, or underline independently or in combination.

Role of Decorator Pattern:

- Each text style is implemented as a decorator that wraps the plain text object.
- Allows flexible layering of styles, e.g., `UnderlineDecorator(BoldDecorator(ItalicDecorator(Text)))`.

- Avoids subclassing for all combinations like `BoldItalicUnderlineText`, keeping the design clean and extensible.

Class Diagram

The class diagram for the Decorator Pattern illustrates the relationship between the component interface, concrete components, and decorators. It shows how decorators extend the functionality of components without modifying their structure.